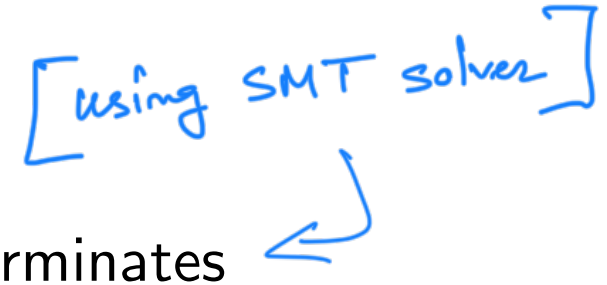
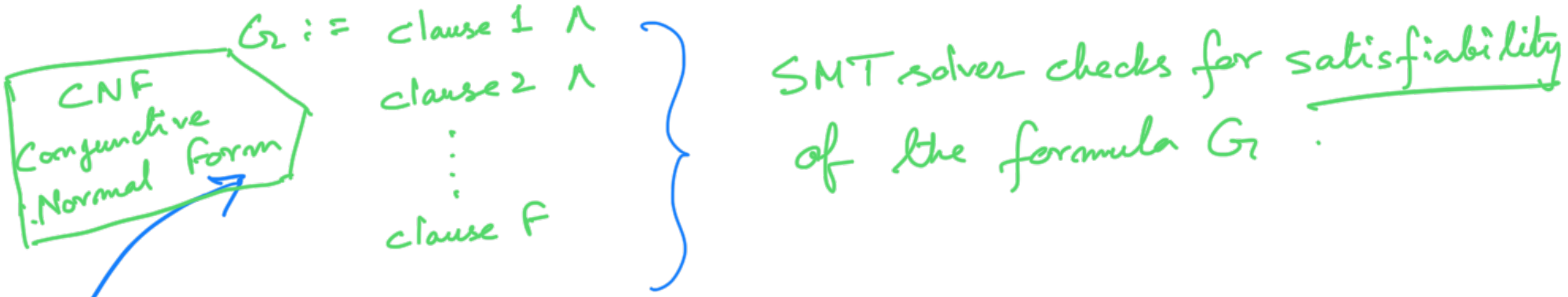
SMT-based Approach – Basic Algorithm

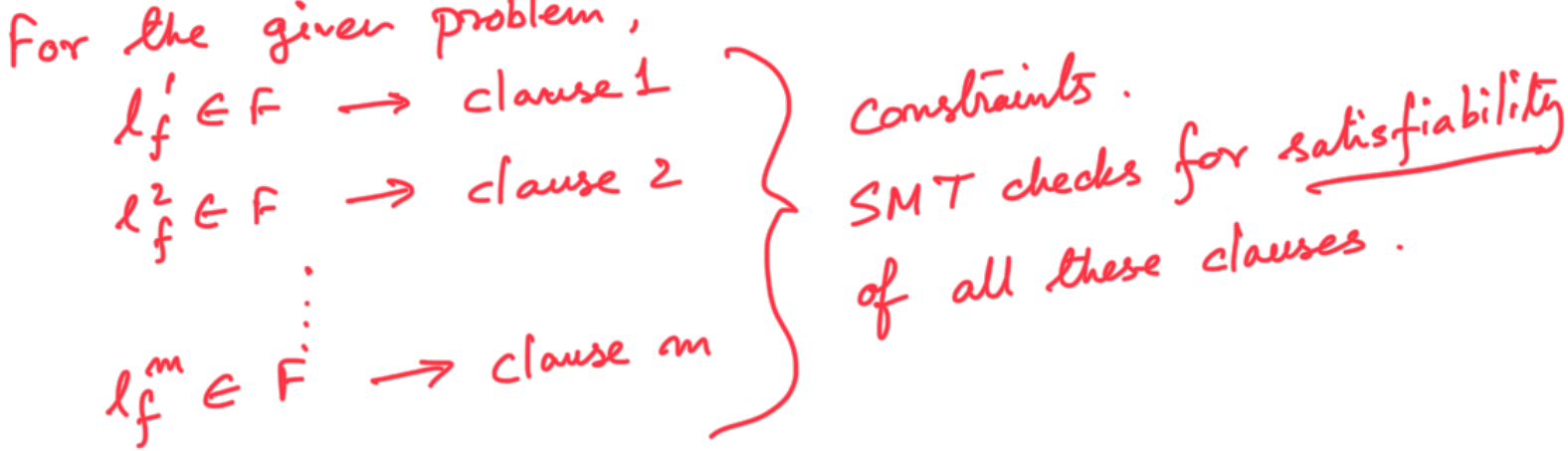
- Simply encodes the constraints
- Provides optimal solution when terminates
- **Scalability issue** with increasing state space (due to larger worksoaces and complex robot dynamics)



SMT-based Approach – UNSAT Core based Algorithm



- **Unsatisfiable core :** In an unsatisfiable propositional formula, a subset of clauses whose conjunction is still unsatisfiable.
- **Unsat-core based algorithm** for FindNCS
  - Significantly increases scalability.



Example of UNSAT-Core based CSP with Turtlebot, $d = 5$ and $\delta = 0$

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

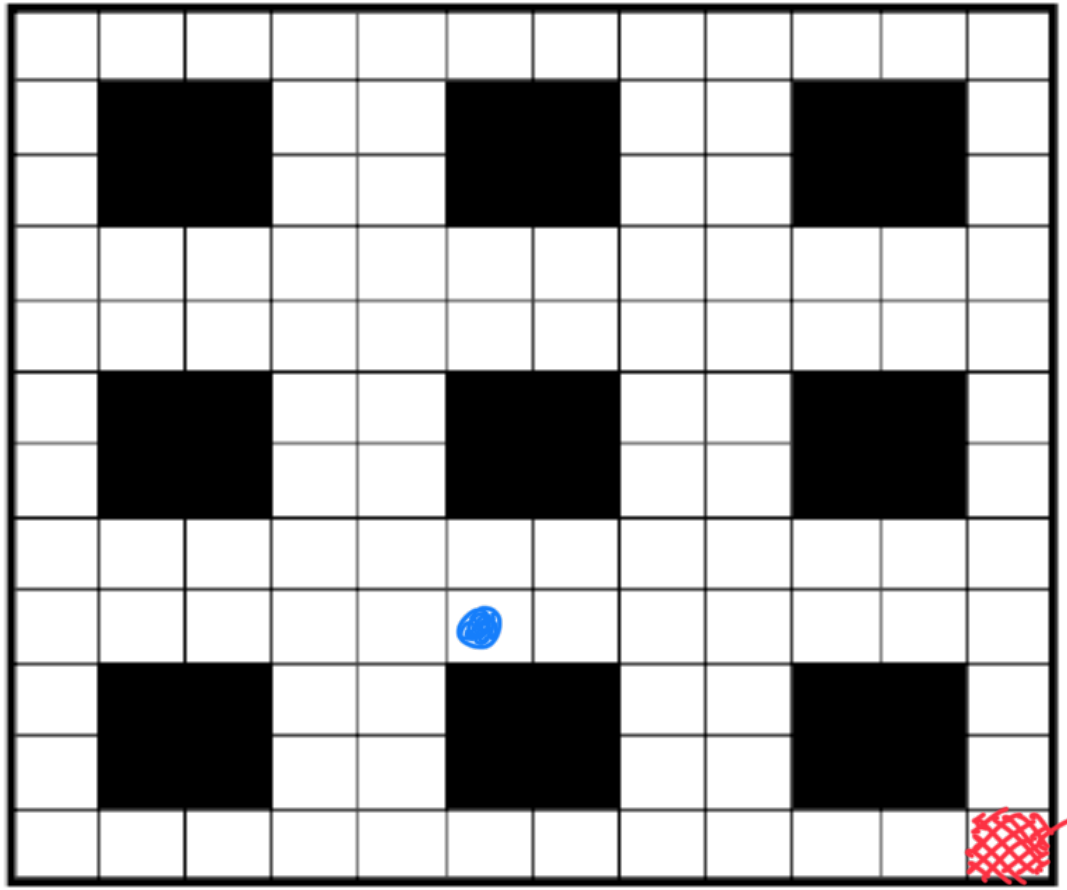


Figure: Check for satisfiability of F with $ncs = 1$.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

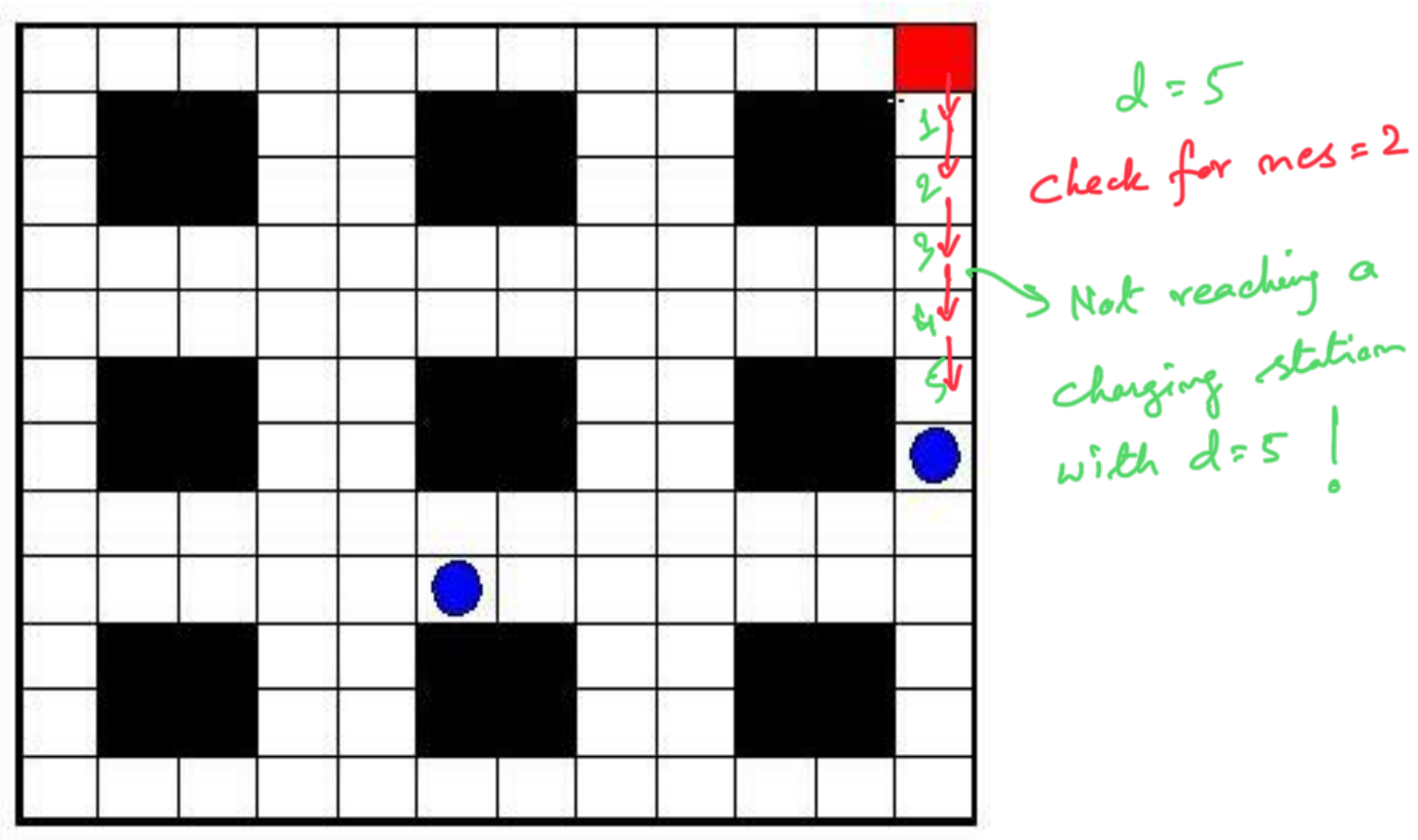


Figure: Check for satisfiability of F with updated CS. SMT solver gives $UNSAT$. Unsat-core (U) contains locations marked in red.

↑ set of obstacle-free locations in the workspace.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

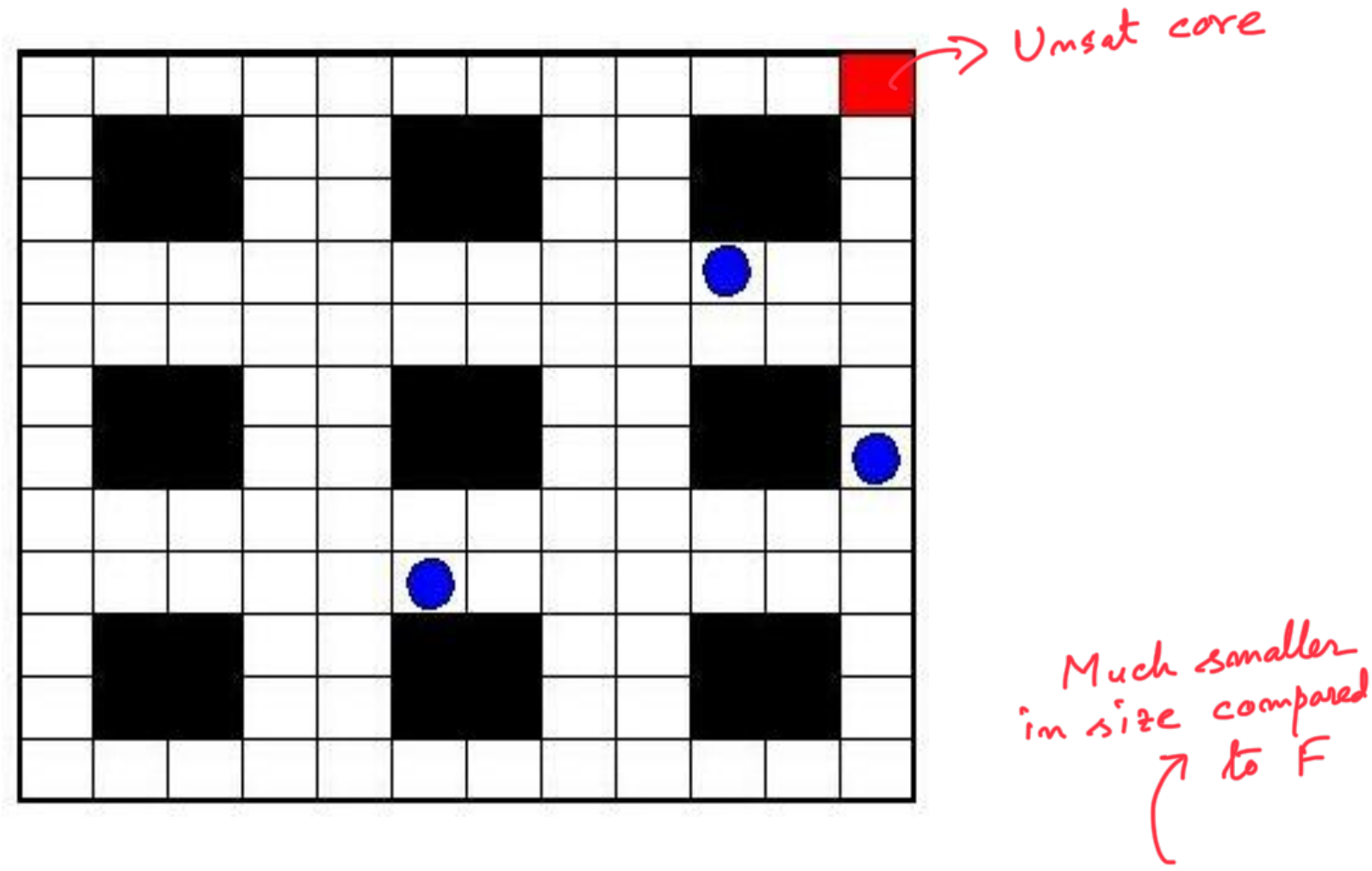


Figure: *ncs* value is increased by 1 and checked for satisfiability of U.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

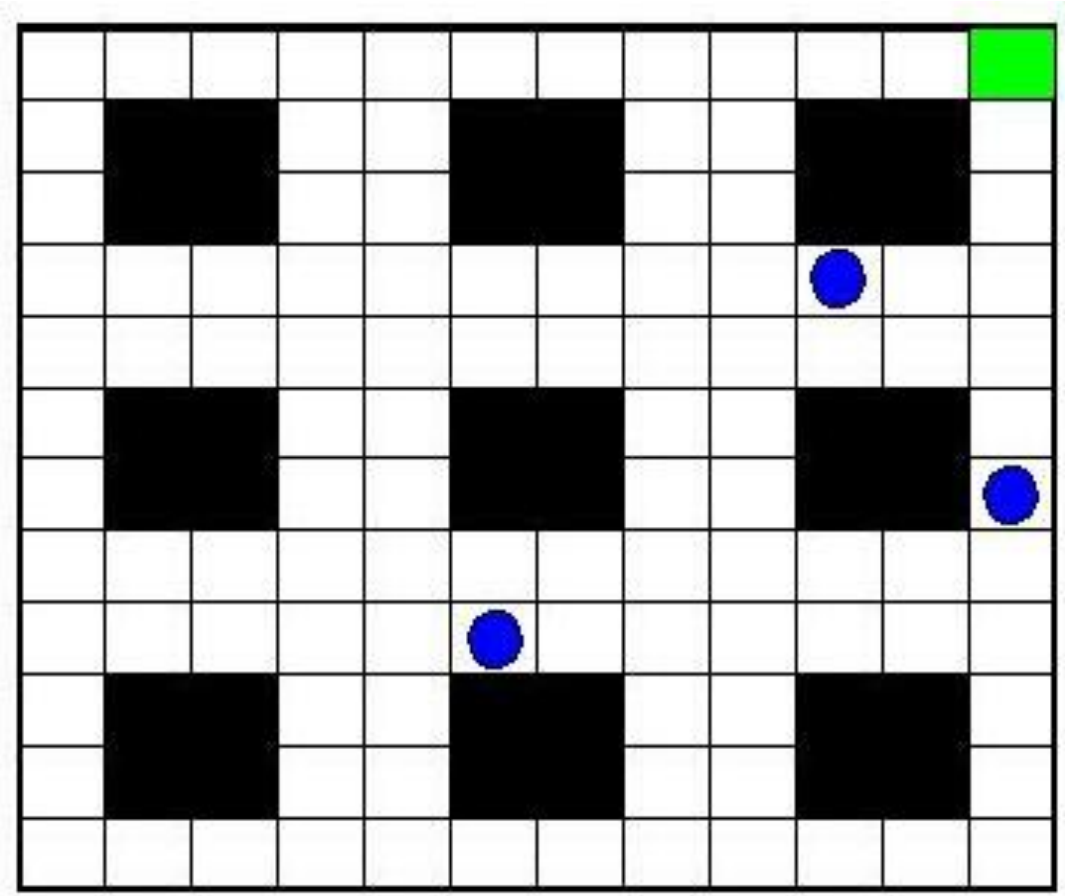


Figure: SMT solver gives *SAT* for U.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

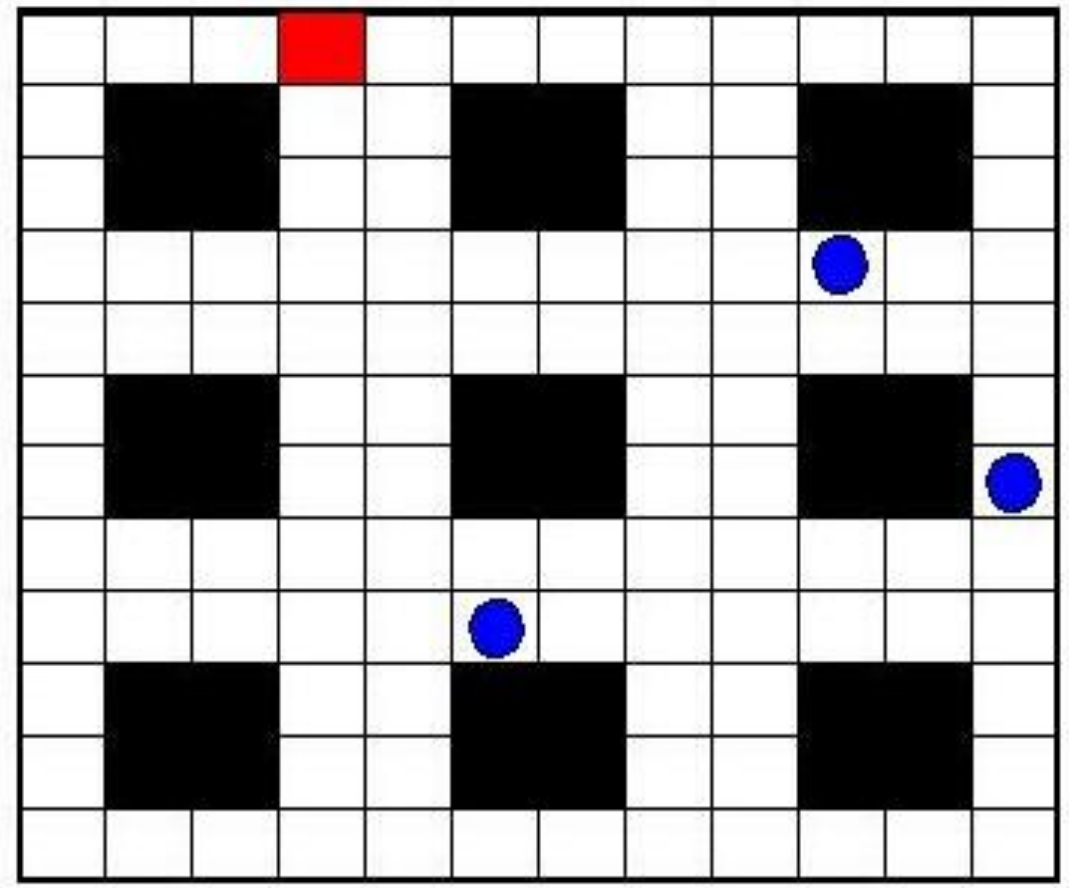


Figure: Check for satisfiability of F with updated CS. SMT solver gives $UNSAT$. Unsat-core (U) contains locations marked in red.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

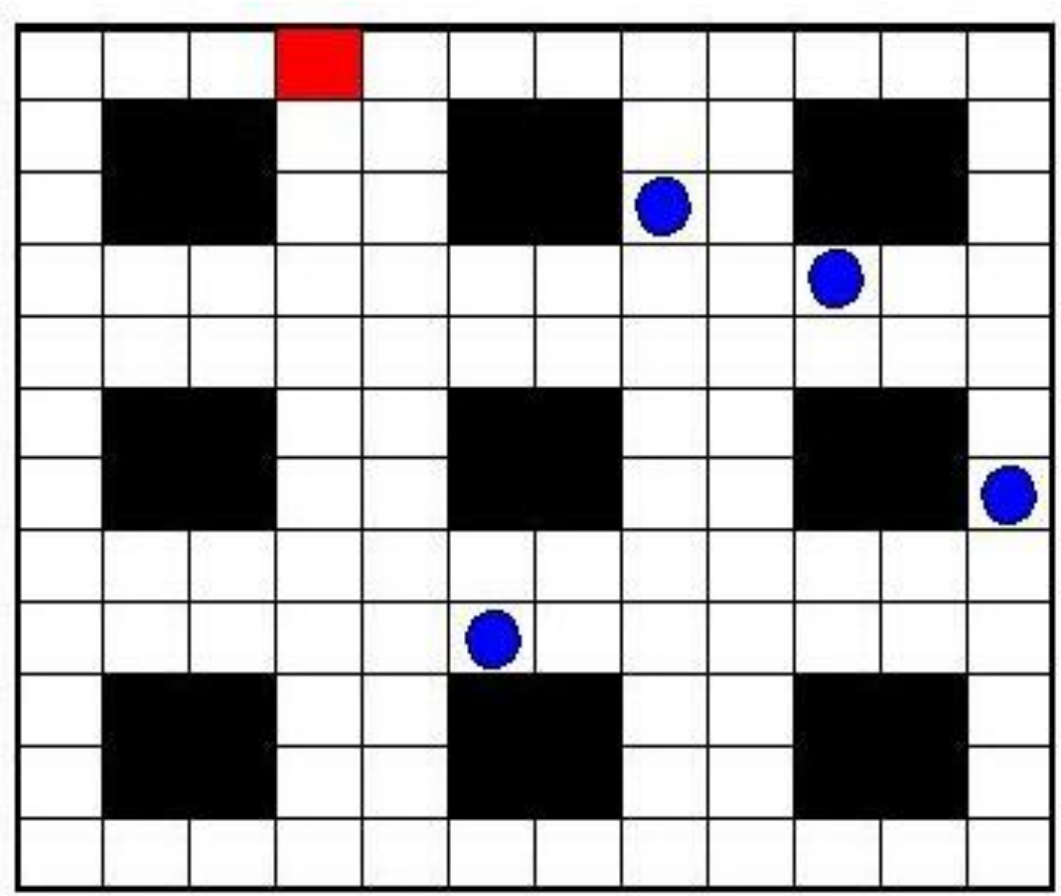


Figure: *ncs* value is increased by 1 and checked for satisfiability of U.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

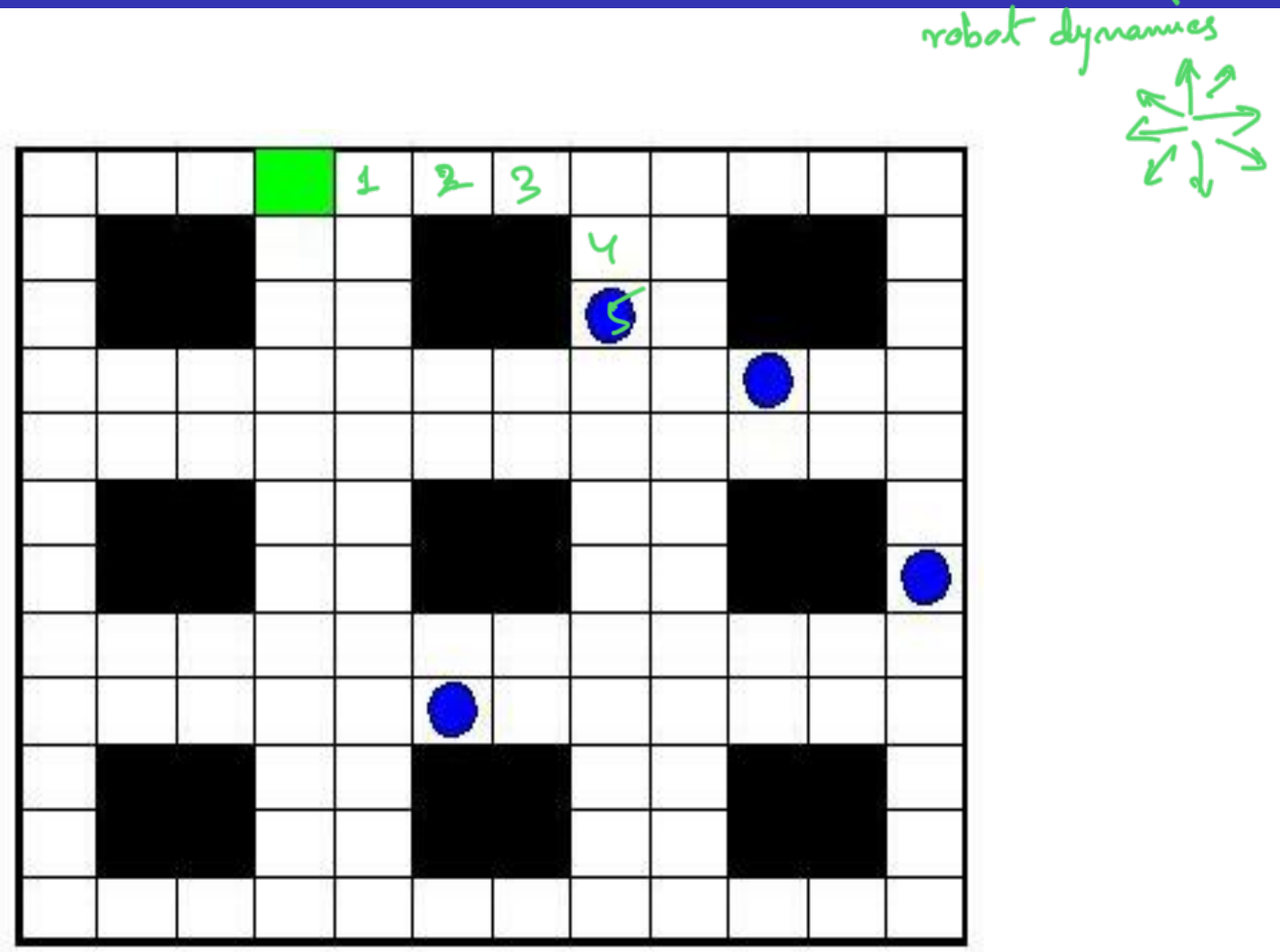


Figure: SMT solver gives *SAT* for U.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

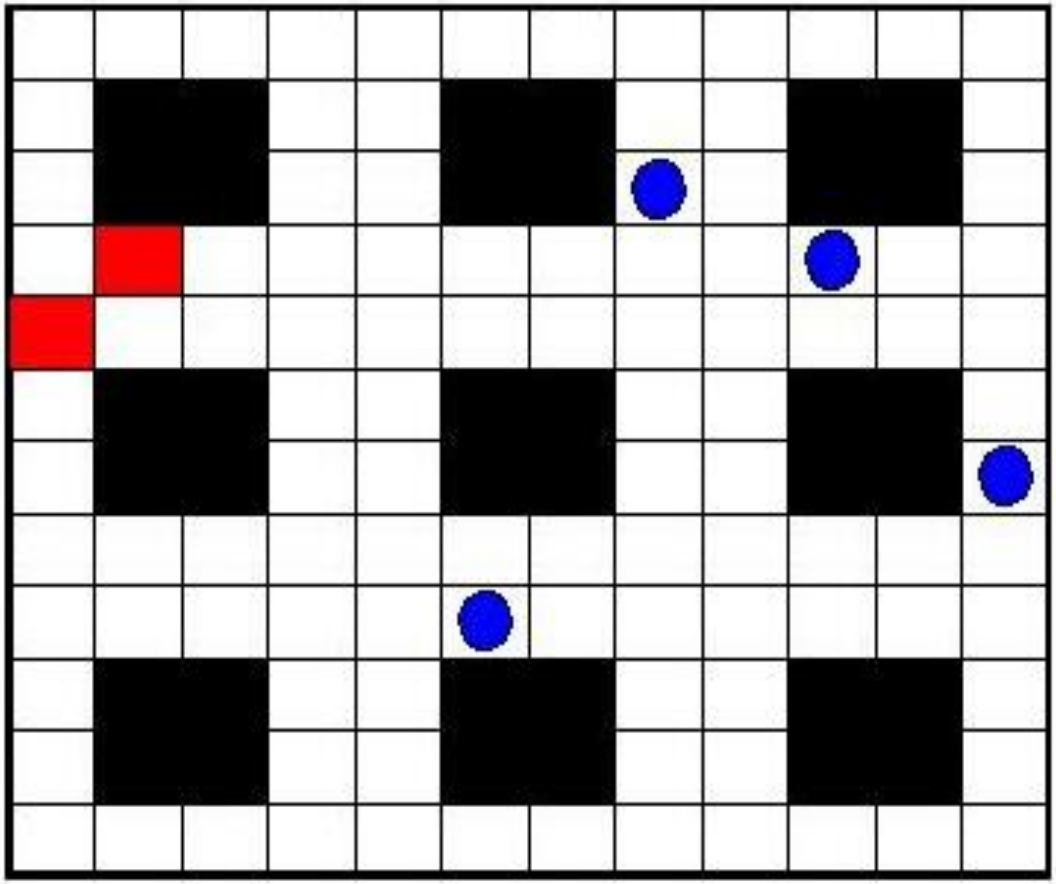


Figure: Check for satisfiability of F with updated CS. SMT solver gives $UNSAT$. Unsat-core (U) contains locations marked in red.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

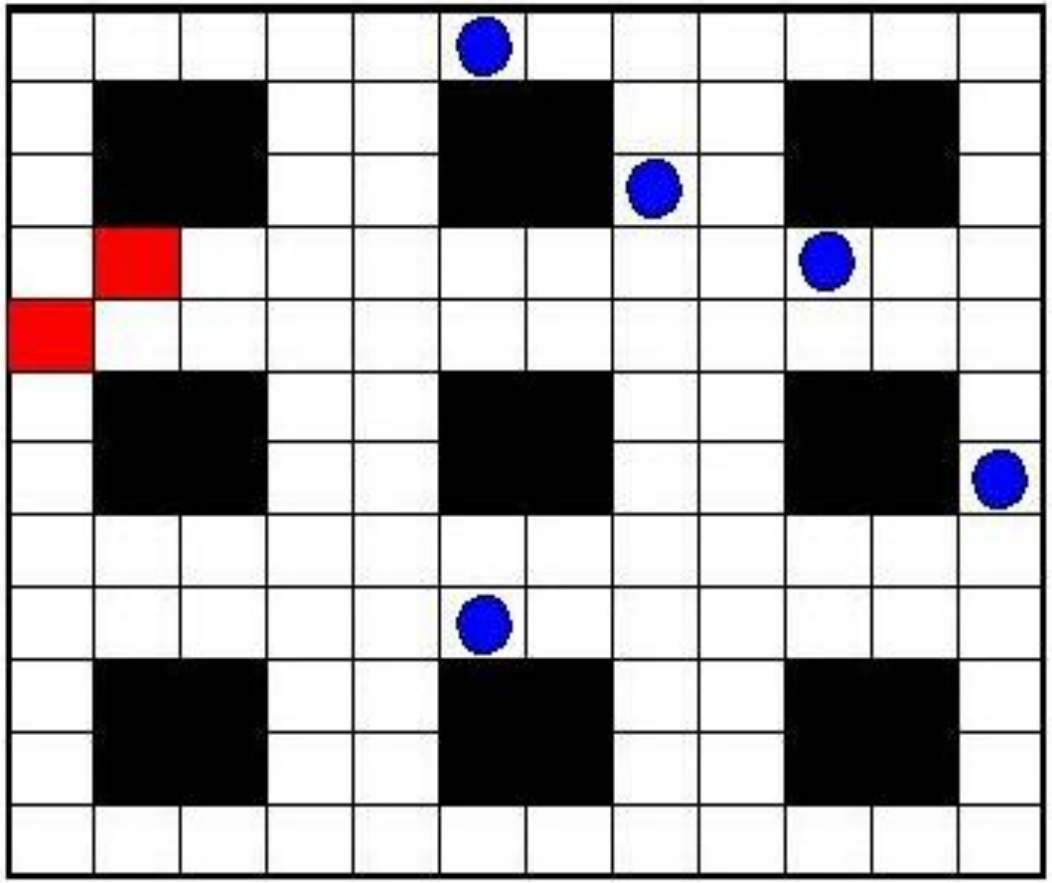


Figure: *ncs* value is increased by 1 and checked for satisfiability of U.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

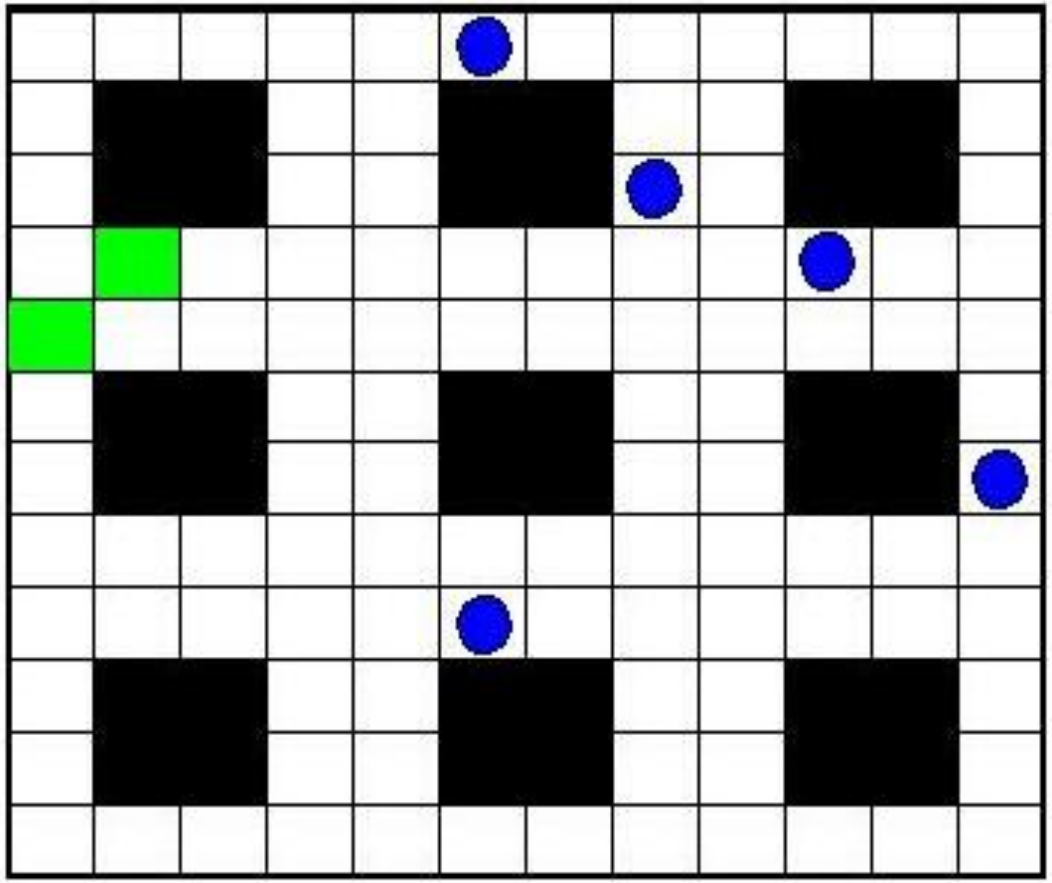


Figure: SMT solver gives *SAT* for U.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 0$

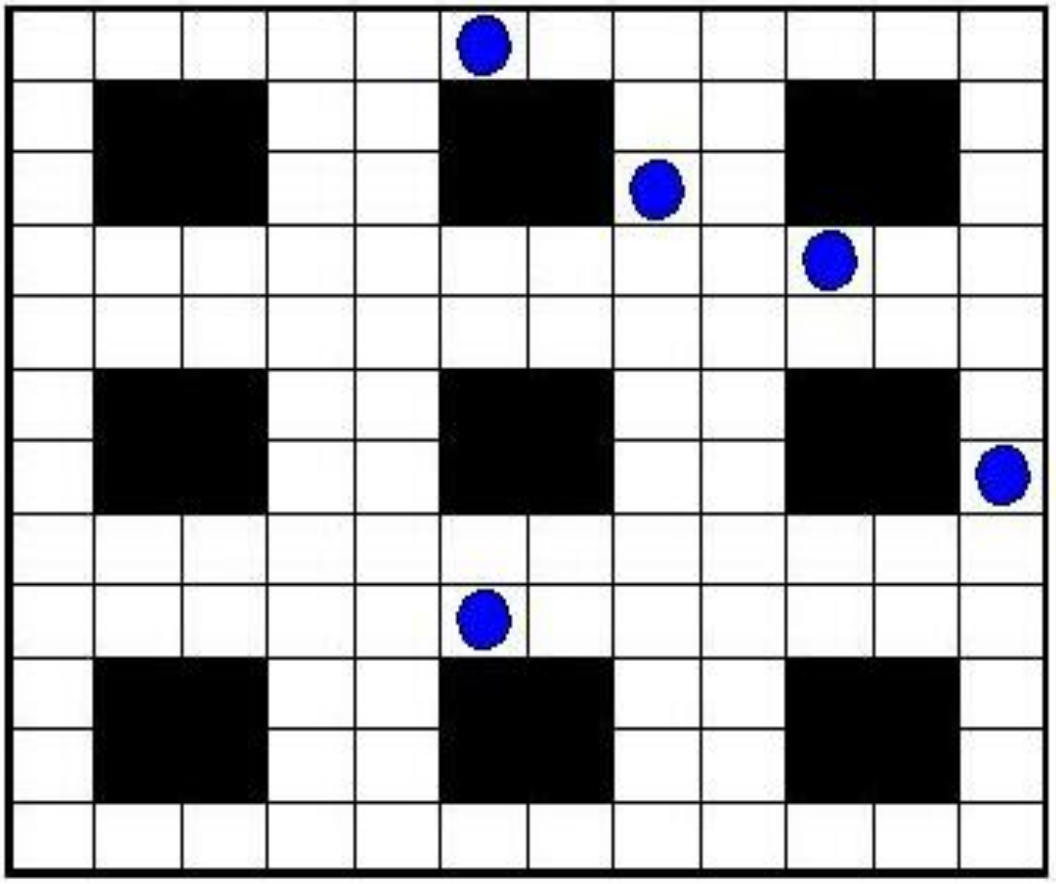


Figure: Check for satisfiability of F with updated CS. SMT solver gives SAT. Return CS.

Example of UNSAT-Core based CSP with Turtlebot, $d = 5$ and $\delta = 3$

δ: controlled perturbation of the CS locations received from the previous iteration

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

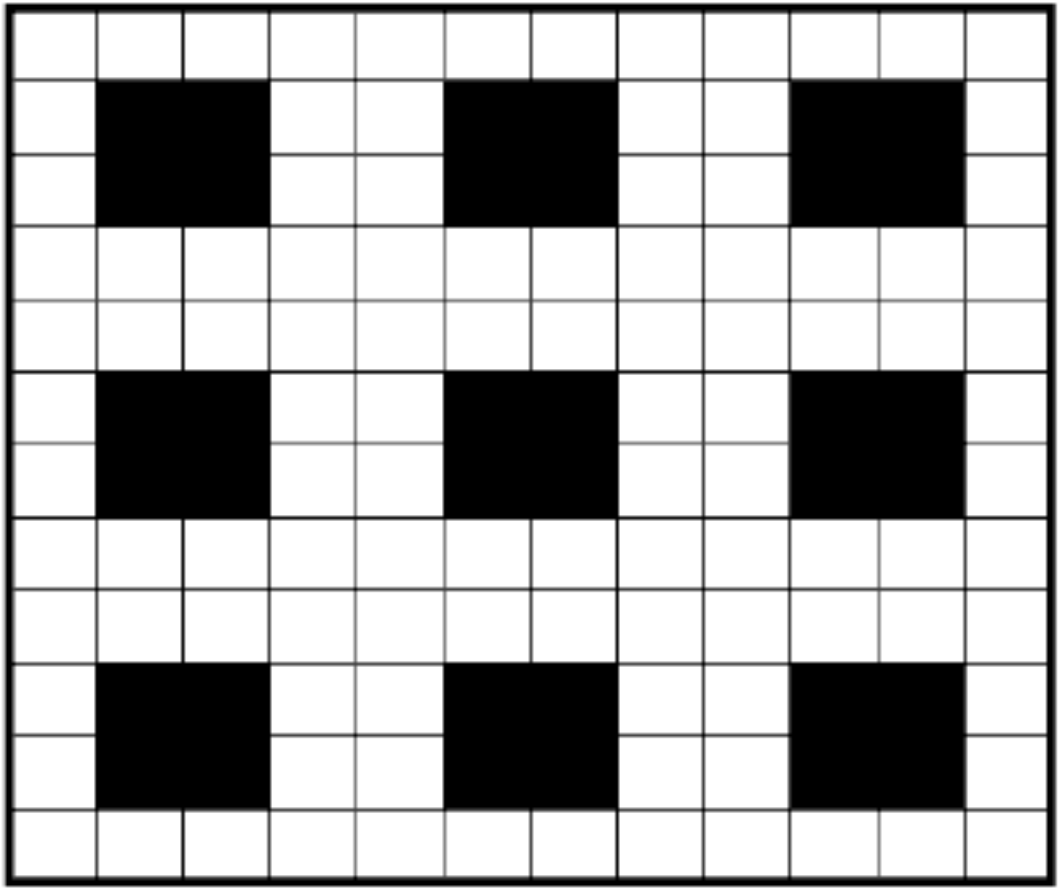


Figure: Check for satisfiability of F with $ncs = 1$.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

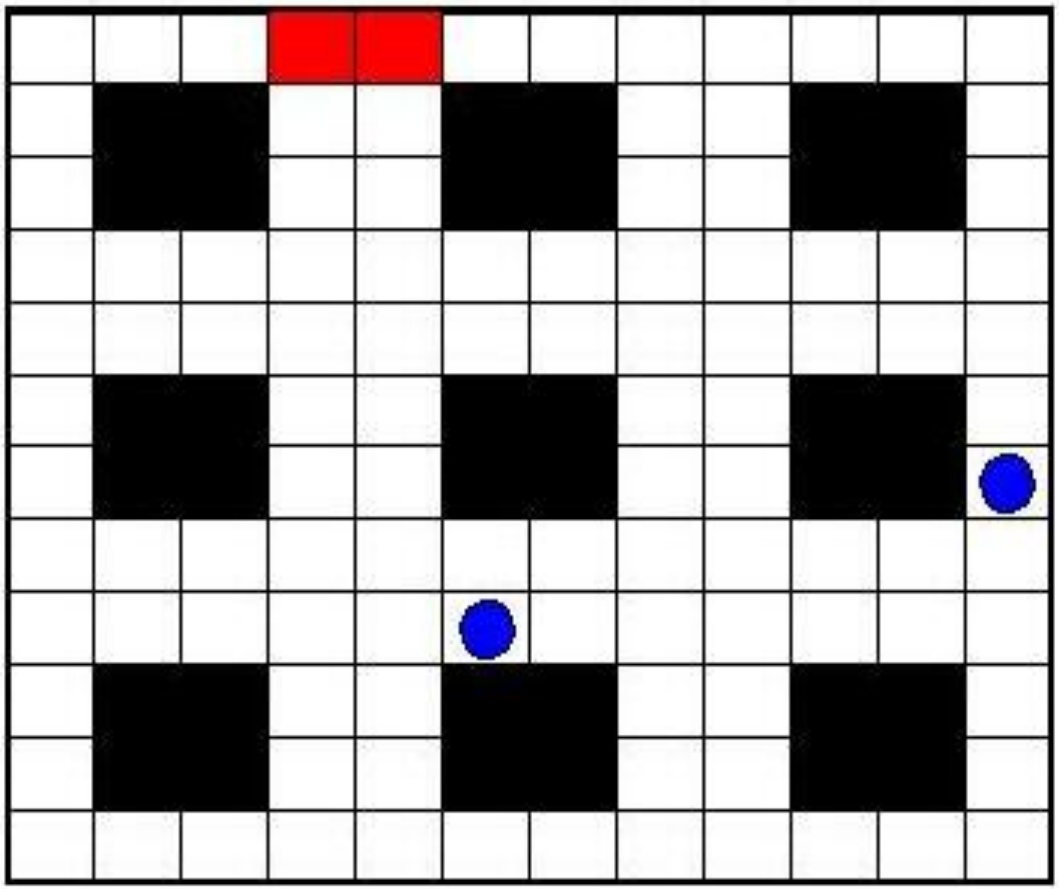


Figure: Check for satisfiability of F with updated CS. SMT solver gives $UNSAT$. Unsat-core (U) contains locations marked in red.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

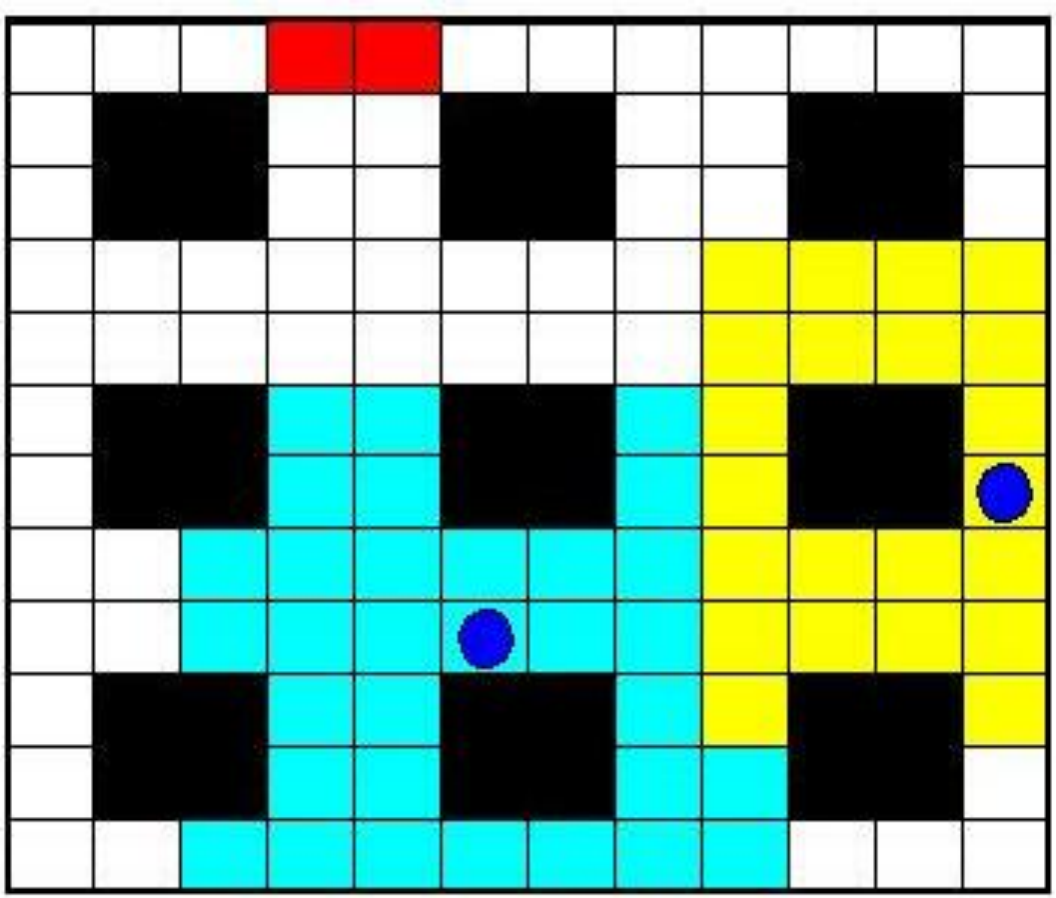


Figure: The locations in CS are perturbed with $\delta = 3$.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

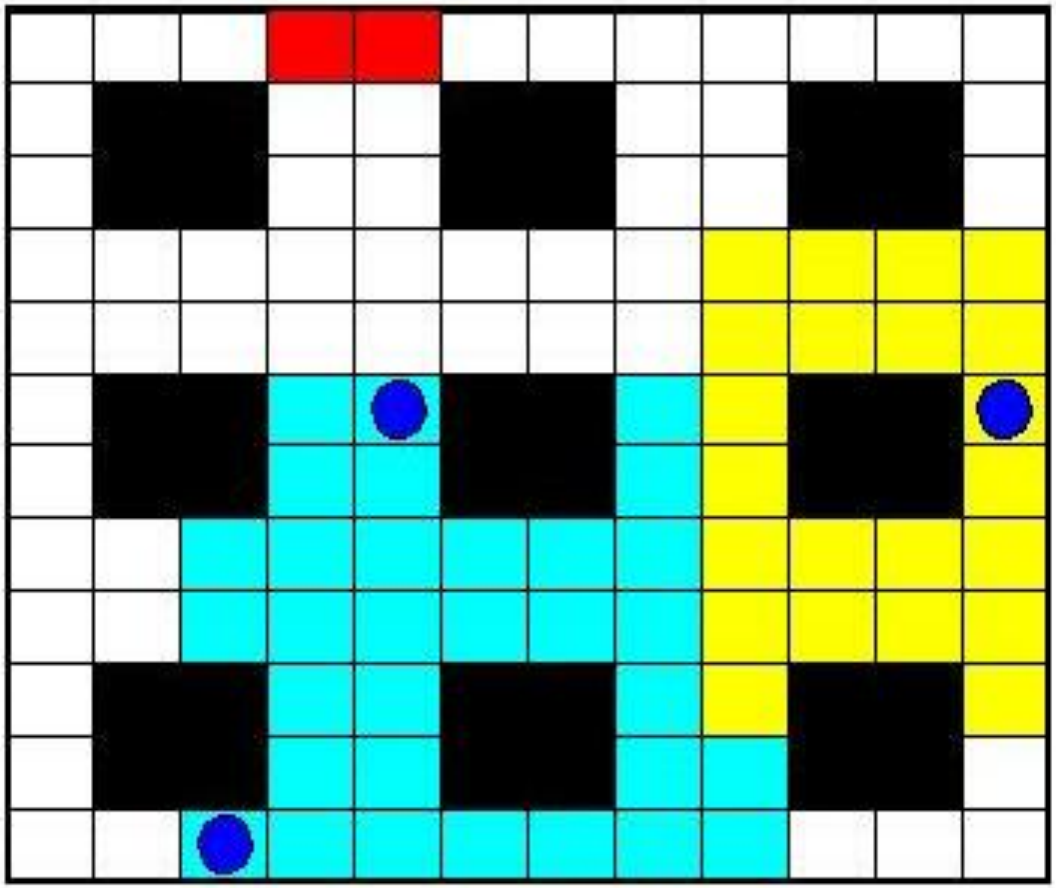


Figure: *ncs* value is increased by 1 and checked for satisfiability of U with perturbed charging station locations.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

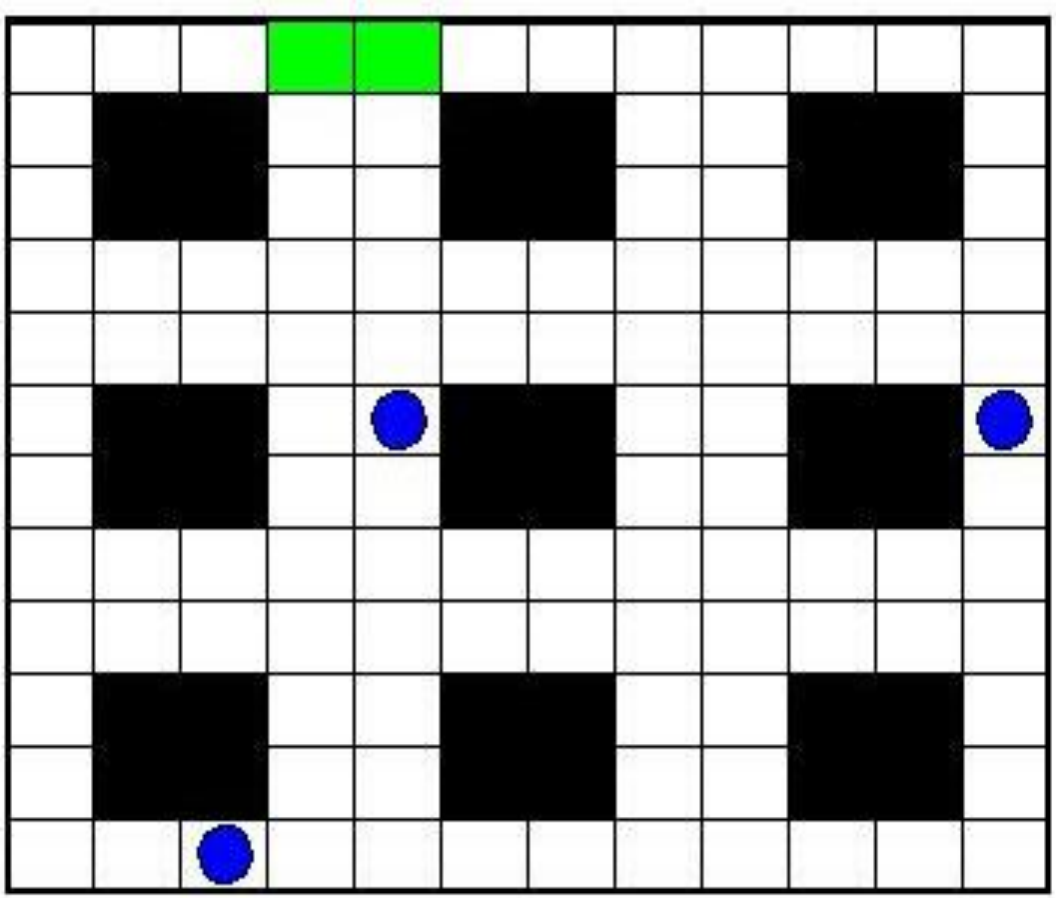


Figure: SMT solver gives *SAT* for U.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

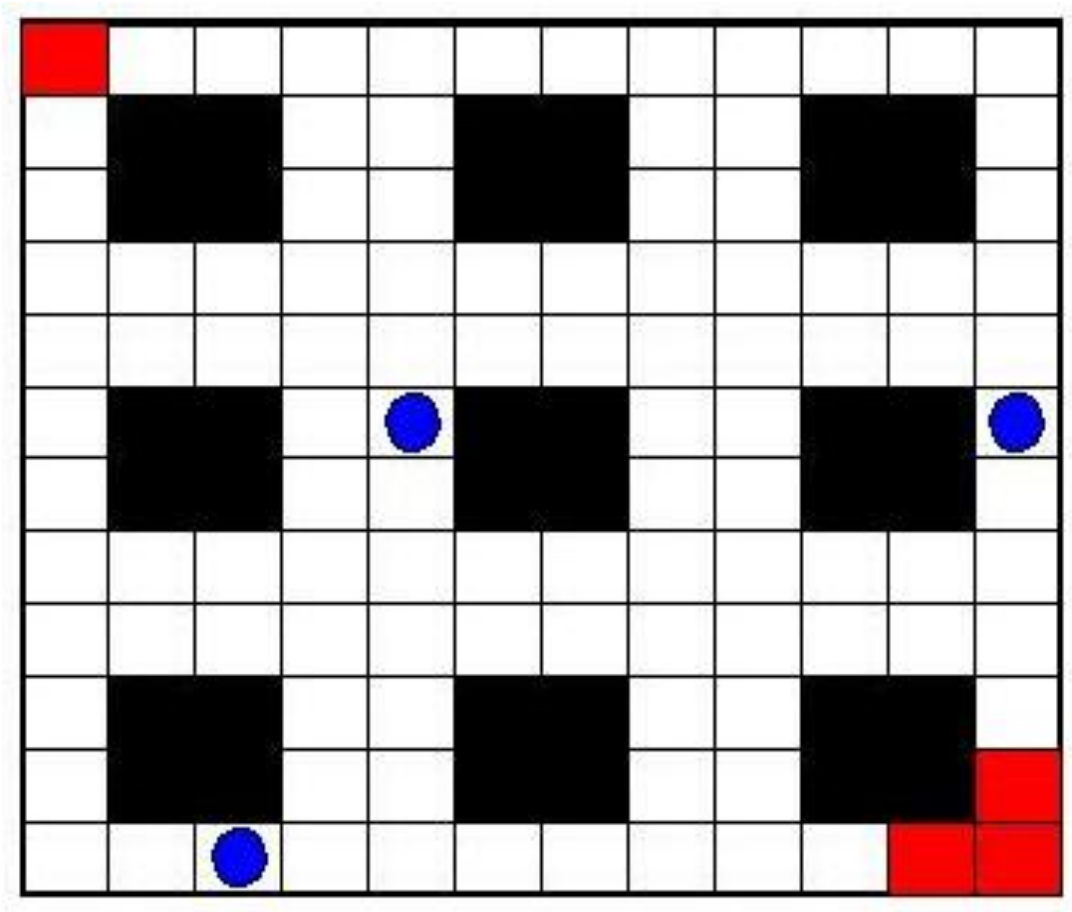


Figure: Check for satisfiability of F with updated CS. SMT solver gives $UNSAT$. Unsat-core (U) contains locations marked in red.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

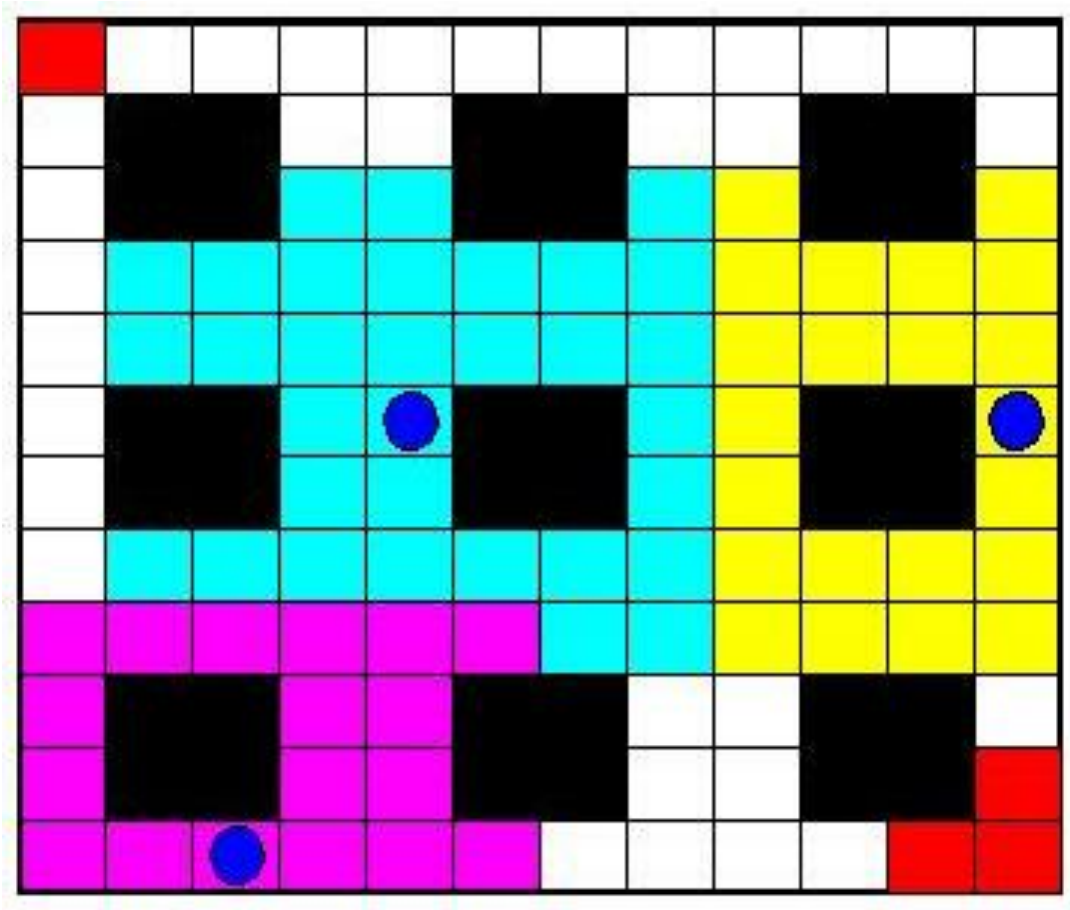


Figure: The locations in CS are perturbed with $\delta = 3$.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

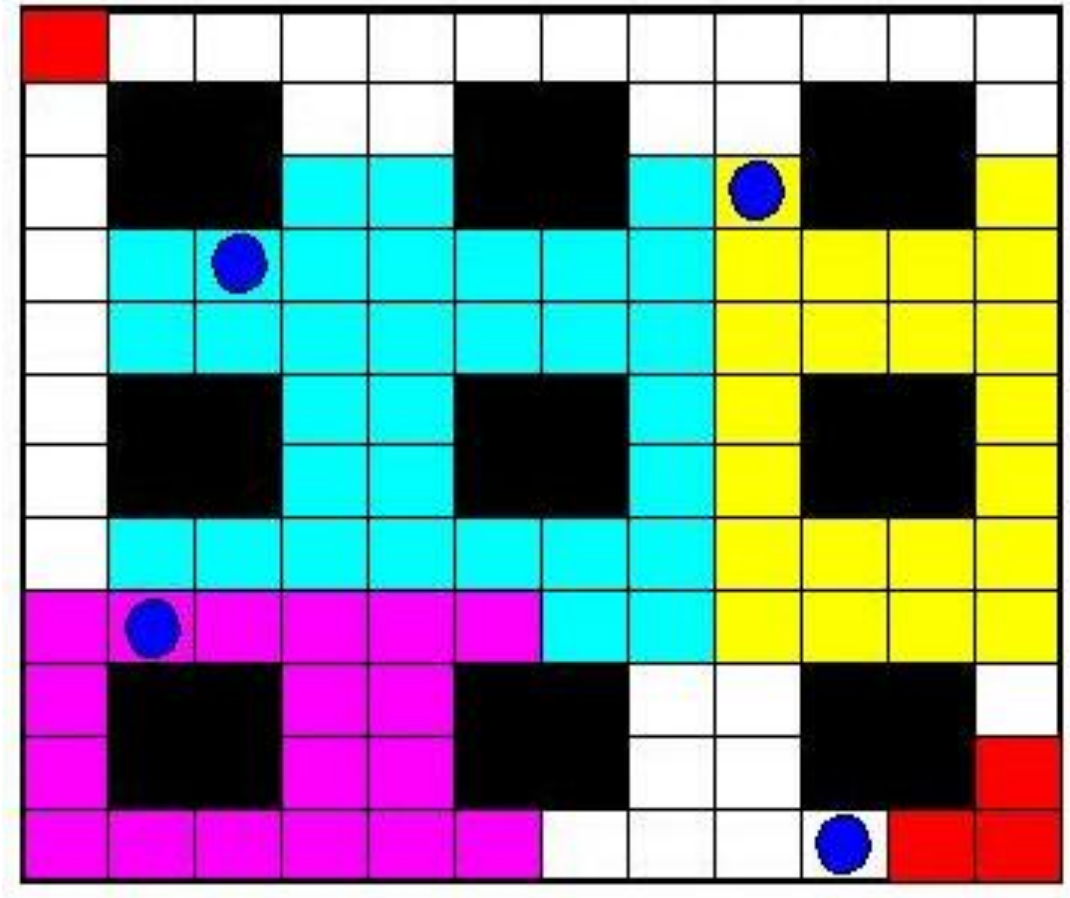


Figure: *ncs* value is increased by 1 and checked for satisfiability of U with perturbed charging station locations.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

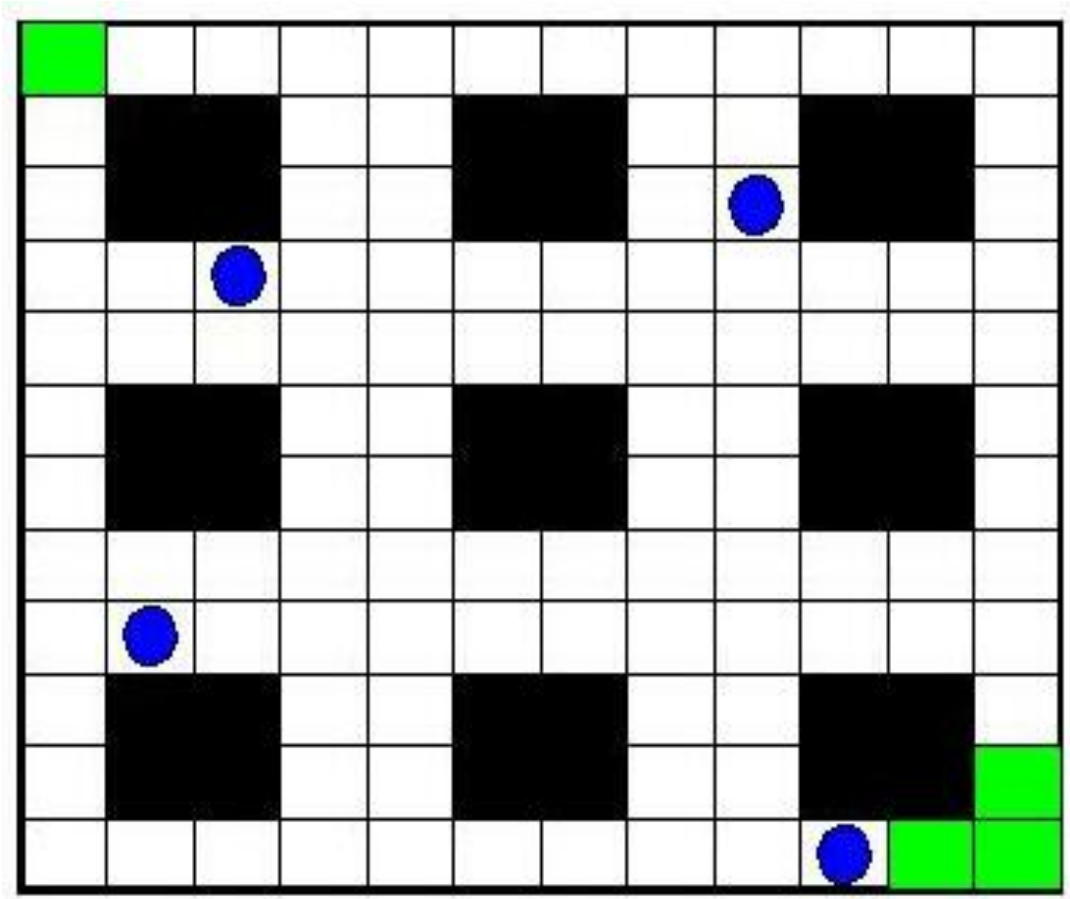


Figure: SMT solver gives *SAT* for U.

Example of UNSAT-Core based CSP with $d = 5$ and $\delta = 3$

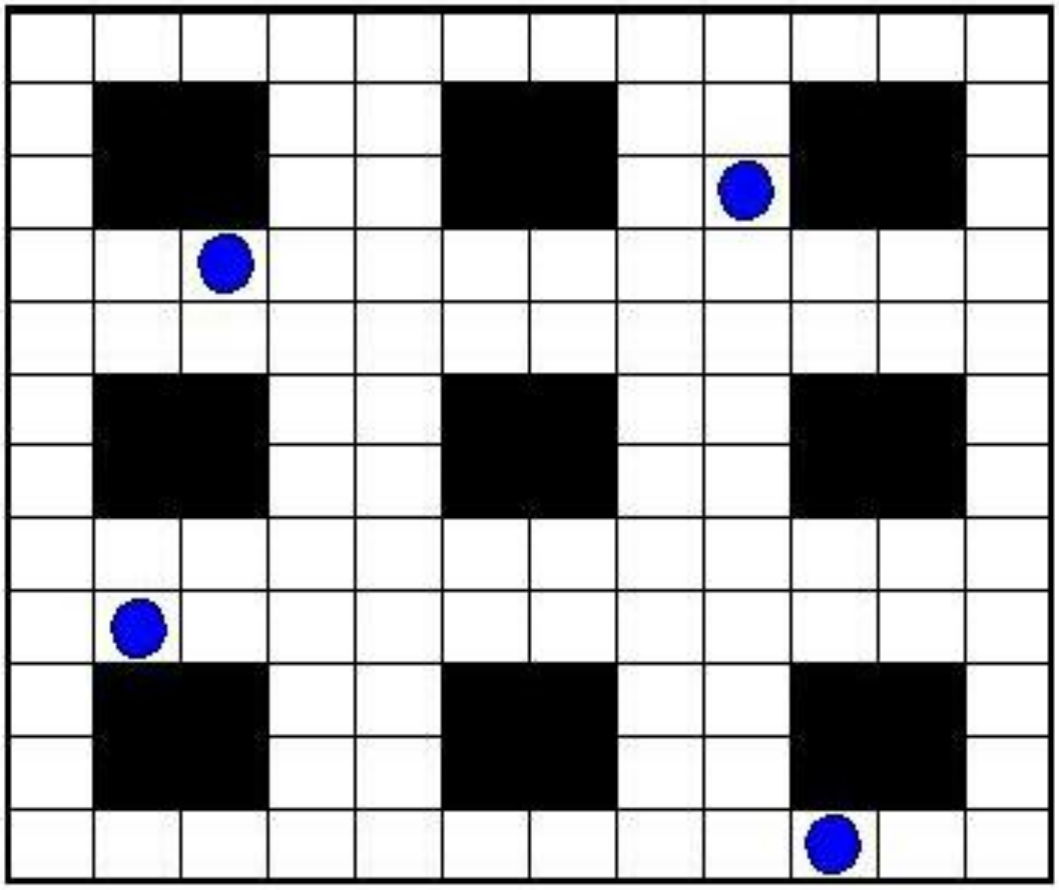
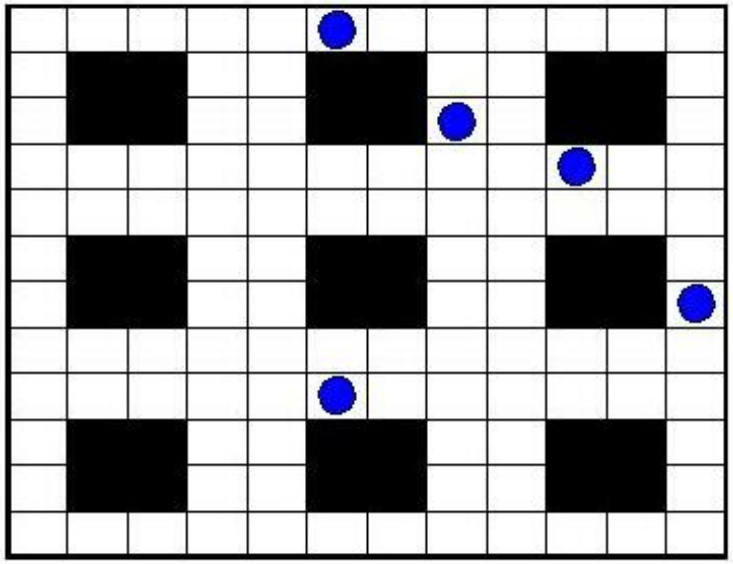
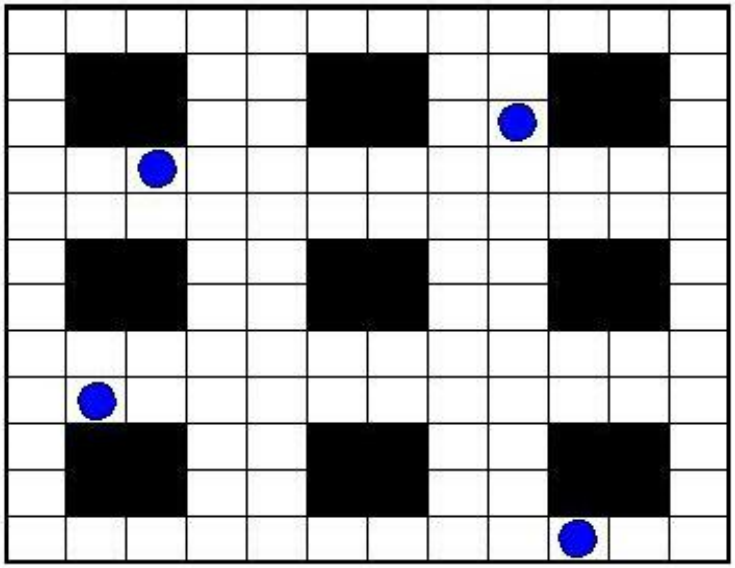


Figure: Check for satisfiability of F with updated CS. SMT solver gives SAT. Return CS.

UNSAT-Core based CSP Comparison with $\delta = 0$ and $\delta = 3$

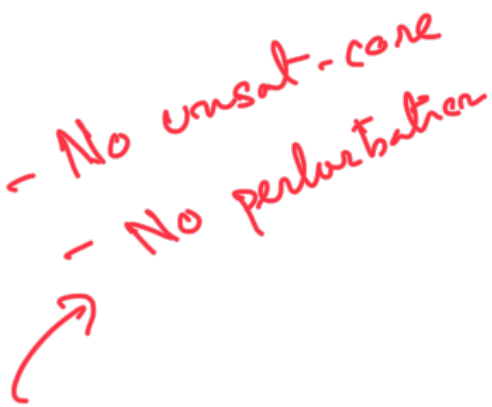


(a) $\delta = 0$



(b) $\delta = 3$

SMT-based Solution – Experimental Results



- **Scalable**: Compared to naive algorithm, our algorithm is up to 23x and 12x faster with $\delta = 0$ and $\delta = 3$ respectively



Thank you